IMAGE INSIGHT VQA

Member 03: UI/UX Design & Gradio Implementation

Project Phase: Final UI Development & Presentation Layer

Document Type: Senior Design + Development Guide

Target Audience: Member 03 Developer

Date: May 17, 2026

Framework: Gradio (Python) + Custom CSS

Design Philosophy: Modern, Interactive, Accessible

DELIVERABLES OVERVIEW

✓ Production-grade Gradio application with intuitive UI ✓ Real-time VQA inference with performance metrics display ✓ Result logging and prediction tracking ✓ Predefined examples for all 6 question types ✓ Professional screenshots for final presentation ✓ Comprehensive handoff documentation ✓ Demo script and backup resources

TABLE OF CONTENTS

1. EXECUTIVE SUMMARY & VISION

Your Role as Member 03

You are responsible for building the final presentation layer—the interface that showcases the VQA model's capabilities to stakeholders, users, and the evaluation team. Your work transforms a working backend into a professional, interactive application. This is the **face of the project**.

Design Vision

Modern Professional + Interactive Intelligence
The UI should feel like a modern AI product—clean, responsive, and intuitive. Users should immediately understand what to do: upload an image, ask a question, get an answer. The interface builds trust through clear feedback, performance metrics, and visual hierarchy.

Key Principles: • Clarity over decoration • Progressive disclosure of information • Real-time feedback for all interactions • Celebration of model intelligence through metrics • Accessible to both technical and non-technical users

2. DESIGN PHILOSOPHY & AESTHETIC DIRECTION

Core Aesthetic: Intelligent Simplicity

This design rejects both sterile minimalism and unnecessary complexity. Instead, we embrace **Intelligent Simplicity**: every visual element serves a purpose, every interaction reveals information, and the design grows in sophistication as users engage deeper.

Think of it as: Apple's design philosophy meets cutting-edge AI product design.

Visual Hierarchy & Spatial Design

Primary Action Zone: Image upload and question input occupy the top-center of the viewport with maximum visual prominence. Clear call-to-action buttons.

Results Zone: Answer, inference time, and question classification appear in a dedicated results panel with subtle animations on reveal.

Secondary Information: Historical examples and performance charts positioned below, easily accessible but not intrusive.

Spacing: Use generous whitespace (padding: 24-32px) between major sections. This creates breathing room and reduces cognitive load.

Motion & Micro-interactions

✓ **Image Upload:** Smooth fade-in with slight scale-up (spring easing) ✓ **Processing State:** Subtle pulsing skeleton loader or animated dots ✓ **Results Reveal:** Staggered animation—answer appears first, then metrics slide in ✓ **Hover States:** Buttons scale slightly (1.02x), background color shifts ✓ **Example Selection:** Quick snap-to with 150ms transition ✓ **Charts:** Animated on first load, bars growing from left to right

3. GRADIO FRAMEWORK ARCHITECTURE

Why Gradio?

Gradio is the ideal choice for this project because:
• Rapid prototyping of ML interfaces (minimal boilerplate)
• Built-in component library (image input, text input, buttons, gallery, etc.)
• Automatic responsive design
• Native support for tabs, blocks, and custom layouts
• Easy integration with existing Python code
• One-line deployment options (HuggingFace Spaces)

Application Structure

File: `app.py`

The main application file contains three logical sections:

1. **Imports & Setup** - Load models, configure logging, define utilities 2. **Core Processing Functions** - Functions that bridge UI inputs to backend logic 3. **Gradio Interface Definition** - Layout, styling, examples, and event handlers

Key Gradio Components You'll Use

Component	Purpose	Properties
gr.Image()	Upload image input	type='pil', interactive=True, label
gr.Textbox()	Question input field	placeholder, lines=2, interactive
gr.Button()	Submit action	value, variant, size, scale
gr.Markdown()	Display answer & metrics	supports HTML, dynamic content
gr.Gallery()	Show example images	columns=3, rows=2, object_fit
gr.Dataframe()	Show example dataset	datatype=['str', 'str', 'str']
gr.Plot()	Render charts	label, value=fig
gr.Tabs()	Organize content	default_tab, label each tab
gr.Blocks()	Custom layout	scale, vertical/horizontal flow

4. COMPONENT SPECIFICATIONS & LAYOUT

Layout Blueprint

Header Section (Top) ■■ Title: "IMAGE INSIGHT: VQA" ■■ Subtitle: "Visual Question Answering Powered by AI" ■■ Small description of the model **Main Input Section (Center-Top)** ■■ Image Upload Component ■ ■■ Drag-and-drop area with icon ■ ■■ Preview of selected image (thumbnail) ■ ■■ Clear/Remove button ■■ Question Input Component ■ ■■ Text area with placeholder ■ ■■ Character counter (optional) ■ ■■ Quick question suggestions (chips) ■■ Submit Button (Large, prominent, color: #0f3460) **Results Section (Center-Bottom)** ■■ Processing Indicator (while running) ■■ Answer Display Panel ■ ■■ Question Type Badge (color-coded) ■ ■■ Predicted Answer (large, bold text) ■ ■■ Inference Time (small, right-aligned) ■ ■■ Confidence Score (if available) ■■ Metrics Panel ■■ Visualization of inference time ■■ Question type accuracy chart (from results/) **Secondary Tabs (Bottom)** ■■ Examples Tab ■ ■■ Gallery of all 6 question types with answers ■■ Predictions Log Tab ■ ■■ Table of all user submissions ■■ About Tab ■■ Model information, limitations, citations

Component Sizing & Spacing

Component	Width	Height	Spacing (margin/padding)
Header	100%	auto	32px bottom
Image Upload	60%	320px	24px padding
Text Input	60%	100px	24px padding
Submit Button	60%	48px	16px top
Results Panel	70%	auto	32px top/bottom
Tabs Container	80%	auto	48px top

5. COLOR PALETTE & TYPOGRAPHY SYSTEM

Color Palette

The color system is based on **Professional Navy + Accent Warmth**.

Primary Color Suite:
• Primary Navy: #0f3460 (buttons, headers, active states)
• Dark Slate: #1a1a2e (text, backgrounds)
• Light Gray: #f9f9f9 (subtle backgrounds, borders)

Accent Colors (Status & Feedback):
• Success Green: #27ae60 (correct answers, positive feedback)
• Warning Orange: #e67e22 (processing, attention needed)
• Error Red: #e74c3c (errors, incorrect answers)
• Info Blue: #3498db (informational messages)

Question Type Color Coding:
• Counting: #9b59b6 (purple - quantitative)
• Yes/No: #2980b9 (blue - boolean)
• Color: #e74c3c (red - visual property)
• Object: #27ae60 (green - entity recognition)
• Action: #f39c12 (orange - dynamic)
• Spatial: #16a085 (teal - spatial reasoning)

Typography System

Font Stack Recommendation:
• Display Font (Headings): 'Segoe UI', 'Helvetica Neue', sans-serif
• Body Font (Text): 'Segoe UI', -apple-system, BlinkMacSystemFont, sans-serif
• Code Font (Metrics, numbers): 'Monaco', 'Courier New', monospace

Font Sizing & Hierarchy:
• Page Title: 32-36px, bold, #1a1a2e
• Section Headers: 20-24px, bold, #0f3460
• Subsection: 16-18px, semi-bold, #16213e
• Body Text: 14px, regular, #2a2a2a
• Small Labels: 12px, regular, #555555
• Input Labels: 13px, semi-bold, #1a1a2e
• Predicted Answer: 28-32px, bold, #0f3460 (highlight!)
• Metrics/Numbers: 14px, monospace, #16213e

Line Heights:
• Body text: 1.6 (comfortable reading)
• Headings: 1.2 (tight, powerful)
• Labels: 1.4 (clean, organized)

6. USER FLOW & INTERACTION PATTERNS

Primary User Journey

1. **Landing** → User arrives, sees clean interface with image upload prominent 2. **Exploration** → User optionally views Examples tab to understand capability 3. **Upload** → User drags/clicks to upload an image ■■ Preview appears with dim overlay, clear button visible 4. **Input** → User types a question in the text area ■■ Placeholder text guides: "Ask a question about the image..." 5. **Submit** → User clicks "Analyze Image" button ■■ Button shows loading state (disabled, spinner/text change) 6. **Processing** → Skeleton loader or animated spinner shows inference in progress ■■ Small text: "Analyzing image and question..." 7. **Results** → Answer appears with animations ■■ Question Type badge slides in (color-coded) ■■ Predicted Answer fades in with emphasis ■■ Inference time appears (e.g., "■ 0.122s") ■■ Optional: Confidence score or model certainty 8. **Continuation** → User can: ■■ Try another image ■■ View their prediction history ■■ Explore different question types

Edge Case & Error Handling

If no image is uploaded: → Display red error box: "Please upload an image first" → Focus highlight on image upload area **If question is empty:** → Display warning: "Please enter a question" → Disable submit button until both fields have content **If inference fails:** → Show error modal: "Model inference failed. Please try again." → Log error to console and predictions.csv → Offer reload or try with different inputs **If image format is invalid:** → Display message: "Please upload a valid image file (JPG, PNG, GIF)" → Clear invalid input

7. CODE IMPLEMENTATION GUIDE

Step 1: Setup & Imports

```
import gradio as gr from PIL import Image import pandas as pd import matplotlib.pyplot as plt from datetime import datetime import os # Import your backend modules from src.model_loader import load_model from src.inference import answer_question from src.question_types import classify_question_type from src.logger import log_prediction from src.preprocessing import preprocess_image
```

Step 2: Initialize Model (Startup)

```
# Load model once at startup (not on every inference) model = None def initialize_model(): global model model = load_model() print("✓ Model loaded successfully") # Call at app launch initialize_model()
```

Step 3: Core Processing Functions

```
def process_vqa(image_input, question_input):
    # Validate inputs
    if image_input is None:
        return ("Error: Please upload image", "", "")

    if not question_input or question_input.strip() == "":
        return ("Error: Please enter question", "", "")

    try:
        answer, inf_time = answer_question(image_input, question_input)
        q_type = classify_question_type(question_input)
        answer_display = f"{answer}"
        time_display = f"Time: {inf_time:.3f}s"
        log_prediction(image_input, question_input, answer, q_type, inf_time)
        return (f"{q_type}\n\n{answer_display}\n\n{time_display}", q_type, answer)

    except Exception as e:
        return (f"Inference failed: {str(e)}", "error", "")

def get_type_color(q_type):
    colors = {'counting': '#9b59b6', 'yes_no': '#2980b9', 'color': '#e74c3c',
              'object': '#27ae60', 'action': '#f39c12', 'spatial_scene': '#16a085'}
    return colors.get(q_type, '#0f3460')
```

Step 4: Load Examples Dataset

```
def load_examples(): '''Load example dataset for gallery''' df =
pd.read_csv('data/vqa_test_set.csv') examples = [] for idx, row in df.iterrows():
examples.append([ Image.open(f"data/sample_images/{row['image_id']}.jpg"),
row['question'], row['answer'], row['question_type'] ]) return examples, df examples_data,
examples_df = load_examples()
```

Step 5: Build Gradio Interface

```
# Define custom CSS for enhanced styling
custom_css = '''
@import url('https://fonts.googleapis.com/css2?family=Segoe+UI:wght@400;600;700');
#title { color: #1a1a2e; margin-bottom: 8px; }
.input-section { background: #f9f9f9; padding: 32px; border-radius: 12px; }
#submit-btn { background: #0f3460; color: white; font-weight: 600; }
'''

with gr.Blocks(css=custom_css, theme=gr.themes.Soft()) as demo:
    gr.Markdown("# IMAGE INSIGHT: VQA")
    gr.Markdown("Visual Question Answering Powered by Advanced AI")

    with gr.Column(scale=1):
        with gr.Group(elem_classes="input-section"):
            gr.Markdown("### Upload & Ask")
            image_input = gr.Image(type="pil", label="Image")
            question_input = gr.Textbox(label="Question",
                                        placeholder="Ask about the image...", lines=3)
            submit_btn = gr.Button("Analyze Image", variant="primary")

        answer_output = gr.HTML(label="Result")

        with gr.Tabs():
            with gr.TabItem("Examples"):
                examples_gallery = gr.Gallery(value=examples_data, columns=3)

            with gr.TabItem("Predictions"):
                predictions_df = gr.Dataframe(value=pd.read_csv('data/predictions.csv'))

            with gr.TabItem("About"):
                about_text = "Model: Salesforce BLIP-VQA (Base)\nAccuracy: 93.8%"
                gr.Markdown(about_text)

    submit_btn.click(fn=process_vqa, inputs=[image_input, question_input],
                     outputs=[answer_output], show_progress=True)

demo.launch(share=True)
```

8. PROMPT ENGINEERING FOR BETTER RESULTS

Why Prompt Engineering Matters

While the model is pre-trained, the way you **frame questions and process inputs** significantly affects answer quality. This section provides strategies to maximize model performance.

Strategy 1: Question Normalization

Problem: Users ask questions in diverse ways ("how many giraffes?", "giraffes count?", "what is the giraffe number?")

Solution: Normalize questions to standard formats before sending to the model.

Implementation:

```
def normalize_question(question): '''Normalize question format for better model performance''' question = question.strip().lower() # Add "how many" if counting indicators present counting_keywords = ['count', 'how many', 'number of', 'total'] if any(kw in question for kw in counting_keywords): if not question.startswith('how many'): question = 'how many ' + question.replace('count', '').replace('how many', '').strip() # Ensure proper capitalization for submission return question.capitalize()
```

Strategy 2: Context-Aware Processing

Key Insight: The model performs better when questions are grammatically correct and natural.

Do's: ✓ "How many people are in the image?" (clear, direct) ✓ "What color is the car?" (specific, concrete) ✓ "Is there a dog in the picture?" (yes/no, unambiguous)

Don'ts: ✗ "ppl here?" (too abbreviated) ✗ "how many like things?" (ambiguous) ✗ "the man with what?" (incomplete/malformed)

Implementation: Add validation to guide users toward better question formats.

Strategy 3: Error Recovery & Fallback Logic

When inference fails or confidence is low:

1. **Retry with normalized question** - Sometimes minor formatting helps 2. **Try alternative phrasing** - Rephrase the question internally 3. **Fallback to simpler model** - Use dandelin/vilt-b32-finetuned-vqa 4. **Return honest uncertainty** - "I'm not confident about this answer"

```
def robust_answer_question(image, question, retry_count=2):
    attempts = [question]

    if 'how many' in question.lower():
        attempts.append(question.replace('how many', 'count'))
```

```
if '?' not in question:
    attempts.append(question + '?')

for attempt in attempts[:retry_count]:
    try:
        answer, inf_time = answer_question(image, attempt)
        if answer and len(answer) > 0:
            return answer, inf_time
    except Exception as e:
        continue

return "Unable to determine answer. Please try different question.", 0.0
```

9. TESTING & VALIDATION CHECKLIST

Pre-Submission Testing

■ **Functional Testing** ■ App launches without errors: python app.py ■ Model loads successfully (check console for "✓ Model loaded") ■ Image upload works (drag & drop + click) ■ Question input accepts text ■ Submit button triggers processing ■ Results display in under 2 seconds ■ Answer text is readable (not overlapped) ■ Question type badge appears with correct color ■ Inference time displays correctly ■ **Test All 6 Question Types** ■ Counting: "How many people are in the image?" ■ Yes/No: "Is there a dog in the image?" ■ Color: "What color is the car?" ■ Object: "What is the main object?" ■ Action: "What is the person doing?" ■ Spatial: "Where is the cat?" ■ **Edge Case Testing** ■ No image selected + submit → shows error ■ Empty question + submit → shows error ■ Invalid image file → shows error message ■ Very large image (>10MB) → handles gracefully ■ Very long question (>500 chars) → truncates or warns ■ Rapid submissions → doesn't crash or queue unexpectedly ■ **User Experience Testing** ■ Interface is intuitive (first-time user can figure it out) ■ Visual hierarchy is clear (know where to start) ■ Results are easy to read ■ No glaring color contrast issues (accessibility) ■ Mobile responsiveness check (if applicable) ■ Example gallery loads and displays correctly ■ Predictions log shows recent submissions ■ Hover states and transitions feel smooth ■ **Performance Testing** ■ Average inference time < 0.2 seconds ■ Memory usage stays below 2GB ■ No memory leaks after 20+ submissions ■ Prediction CSV updates correctly ■ **Visual Polish** ■ No typos in UI text or labels ■ Consistent spacing and alignment ■ Font sizes are appropriate ■ Colors match the specification ■ Button states are clear (hover, active, disabled) ■ Loading indicator is visible and smooth

10. SCREENSHOTS & DEMO PLANNING

Required Screenshots

Capture the following screenshots to include in your final presentation: **1. Landing State** • Empty app, interface ready for input • Shows title, subtitle, upload area, input field, submit button • File: report/screenshots/01_landing_state.png **2. Image Upload Complete** • Image selected and previewed • Question field has focus • File: report/screenshots/02_image_uploaded.png **3. Processing State** • Submit button disabled/loading • Spinner or "Analyzing..." text visible • File: report/screenshots/03_processing.png **4. Results Display (Counting)** • Question type badge: "COUNTING" (purple) • Answer displayed prominently • Inference time shown • File: report/screenshots/04_result_counting.png **5. Results Display (Yes/No)** • Question type badge: "YES_NO" (blue) • Clear yes/no answer • File: report/screenshots/05_result_yesno.png **6. Results Display (Color)** • Question type badge: "COLOR" (red) • Color word as answer • File: report/screenshots/06_result_color.png **7. Examples Tab** • Gallery view showing all examples • Multiple question types visible • File: report/screenshots/07_examples_tab.png **8. Predictions Log Tab** • Table showing submission history • Multiple rows of predictions • File: report/screenshots/08_predictions_log.png **9. Mobile View (if responsive)** • Interface on narrow viewport • File: report/screenshots/09_mobile_view.png **10. Error Handling** • Error message displayed • File: report/screenshots/10_error_state.png

Demo Script (Live Presentation)

Opening (30 seconds) "This is IMAGE INSIGHT, a Visual Question Answering system. It takes an image and a natural language question, and predicts the answer using AI. Let me show you how it works." **Demo Flow (2-3 minutes)** 1. Show the clean interface "The interface is designed to be intuitive. You upload an image and ask a question." 2. Upload an image (from data/sample_images/) "I'll select a diverse image..." 3. Ask a counting question Question: "How many people are in this image?" "Our model correctly identifies... [result]. See the inference time—only 0.12 seconds!" 4. Ask a color question Question: "What color is the person's shirt?" "And here it predicts the color accurately." 5. Ask a spatial question Question: "Where is the dog in the image?" "Notice the question type is detected automatically, and the model reasons about spatial relationships." 6. Show the Examples tab "These are all 48 test examples, covering 6 different question types. Each one was carefully curated for evaluation." 7. Show Predictions Log "The system logs every prediction, so we can track accuracy and analyze failure cases." 8. Closing "Overall accuracy on this dataset: 93.8%. The model excels at object recognition, yes/no questions, and spatial reasoning. Our main challenge is counting—with >5 objects, accuracy drops slightly. This is typical for current VQA models."

11. KNOWN LIMITATIONS & FAILURE CASES

Model Limitations (from Member 02 Evaluation)

Counting (75% accuracy) • Predicted: 3 giraffes | Ground Truth: 2 giraffes • Predicted: 8 pictures | Ground Truth: 7 pictures → Issue: Model tends to overcount when objects are clustered → Recommendation: Ask "approximately how many?" for better results **Color (87.5% accuracy)** • Predicted: red church | Ground Truth: brown church → Issue: Difficulty with subtle color distinctions → Recommendation: Use more specific color descriptions **Overall Failure Rate: 6.2% (3 out of 48 test cases) Why These Failures Occur:** 1. **Model Training Data Bias:** COCO dataset has more indoor than outdoor scenes 2. **Ambiguous Questions:** Questions like "what is this?" are inherently ambiguous 3. **Visual Complexity:** Cluttered backgrounds confuse the model 4. **Domain Shift:** Our test images might differ from COCO distribution

How to Handle Failures in the UI

When users encounter failures, the UI should: 1. **Display the answer honestly** - Don't hide wrong answers 2. **Provide context** - "This model struggles with counting objects > 5" 3. **Suggest alternatives** - "Try asking 'approximately how many?'" 4. **Log for analysis** - Record all failures for future improvement 5. **Offer feedback mechanism** - Let users report wrong answers **Implementation:** Add a "Report Incorrect Answer" button in results panel. This helps us understand model weaknesses.

12. FINAL SUBMISSION CHECKLIST

■ **Code & Implementation** ■ app.py is complete and runs without errors ■ All imports are included ■ Model loads on startup (not on every inference) ■ Gradio interface is visually polished ■ CSS is applied correctly ■ All components are responsive ■ **Testing & Validation** ■ python app.py executes successfully ■ python scripts/validate_dataset.py passes ■ Tested all 6 question types (at least 1 example each) ■ Tested error cases (no image, no question) ■ Verified inference time is acceptable ■ Predictions are logged to CSV ■ **Documentation** ■ handoff/member03_to_team.md is complete ■ Includes setup instructions ■ Lists any dependencies needed ■ Documents any known issues ■ Provides contact/troubleshooting info ■ **Deliverables** ■ Report directory contains: ■■ screenshots/ folder with all 10 screenshots ■■ metrics_summary.csv (from Member 02) ■■ question_type_accuracy.png (from Member 02) ■■ inference_time_chart.png (from Member 02) ■ data/predictions.csv contains recent predictions ■ data/sample_images/ has all 34 images ■ **Presentation Readiness** ■ Demo script is practiced and timed ■ Live demo has backup plan (screenshots if live fails) ■ Can explain architecture and design choices ■ Aware of model limitations and can discuss them ■ Ready to answer questions about trade-offs ■ **Quality Assurance** ■ No typos or grammatical errors in UI ■ All visual elements are aligned and polished ■ Performance is optimized (no lag, fast inference) ■ Accessibility is considered (color contrast, readable fonts) ■ Mobile-friendly (if applicable) ■ **Final Git Actions** ■ Pulled latest main before final commit ■ All changes committed to main branch ■ Commit message is descriptive ■ Pushed to remote repository ■ Verified changes appear on GitHub

Congratulations! You're ready for final submission.